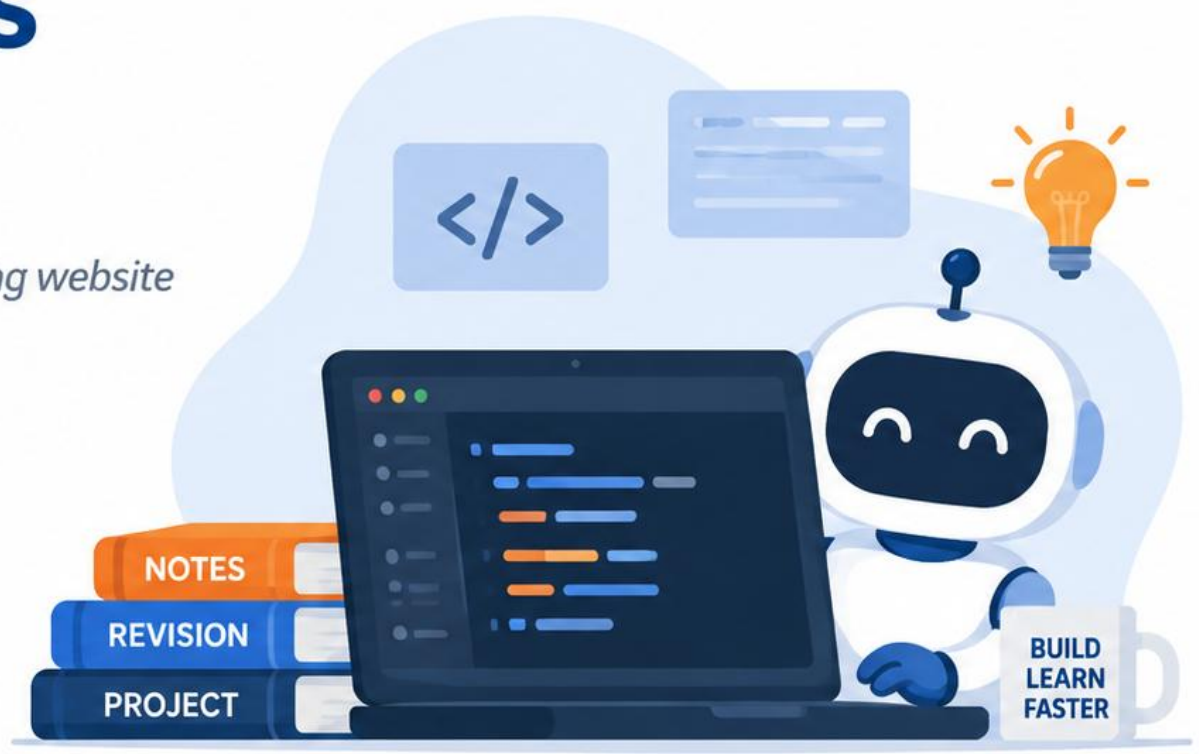
NUS National University of Singapore

NUS BUSINESS ANALYTICS CENTRE

AI Coding Agents Workshop

Claude Code & Codex: notes, revision plans and a working website

CLAUDE CODE



Soumya Haridas & Bhargav Sagiraju

A hands-on session for MSBA batch · September 2026

TODAY'S AGENDA

One story, four acts

Ten sections, about 105 minutes, including a 25-minute live demo

ACT I · SET THE SCENE

01 **AI coding agents 101** *6 min*
What an LLM does, and what Claude / GPT actually are

02 **Claude Code & Codex** *8 min*
Two terminal AI teammates — and what a harness is

03 **Model vs. harness** *6 min*
The shift from writing code to directing it

04 **Getting set up** *8 min*
Install both, run your first prompt

ACT II · YOUR PROJECT

05 **Today's build: ComputeMarket 5 min**
The app you'll leave with

06 **Project files & reusable prompts** *12 min*
AGENTS.md / CLAUDE.md, Skills, permissions

ACT III · LEVEL UP

07 **Skills and connected tools 24 min**
Models, Skills, permissions, MCP and project context

08 **Choosing a working environment 6 min**
Local files, connected apps and cloud tasks

ACT IV · SHOWTIME

09 **LIVE BUILD** *25 min*
A marketplace frontend with task posting, bids and reviews

10 **Recap, ideas & next steps** *5 min*
Ladder for going further on your own

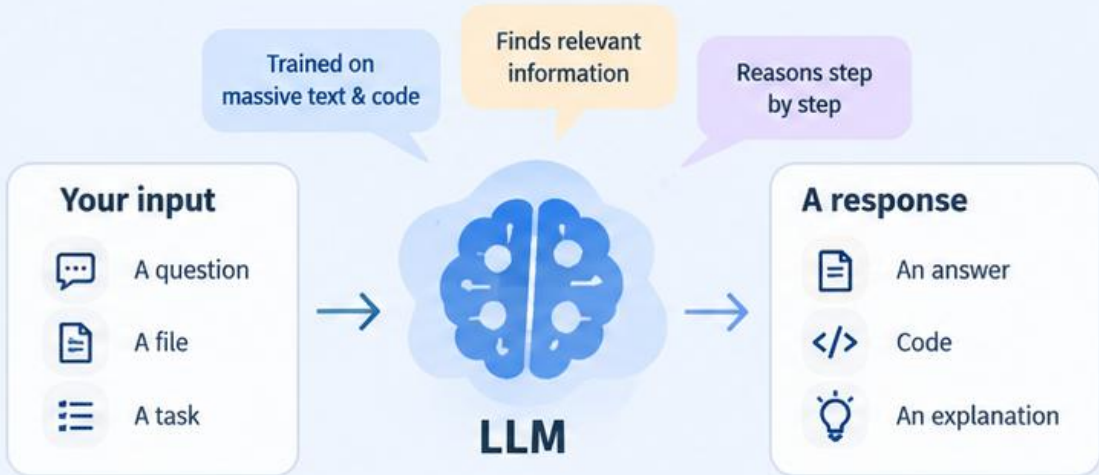
SECTION 01

What's actually under the hood?

The models that power both tools you'll use today

In plain terms

A large language model (LLM) is trained on huge amounts of text and code. You give it input — a question, a file, a task — it reasons over it, and produces a response, one token at a time.



A model generates output from the context available to it. The surrounding product decides which messages, files and tool results to include. Long contexts can increase latency and usage, but caching and compaction change the cost.

Claude and GPT are model families. Claude Code and Codex combine models with tools that can read files, make edits and check results.

01 Writes & reviews code

Python, JavaScript, and most mainstream languages, from a natural-language prompt.

"Refactor this function to handle edge cases and add tests."



02 Reasons over context

Reads a syllabus, a codebase, a PDF — and explains it back in plain language.

"Summarise this paper and highlight the key findings."



03 Uses tools

Uses the tools enabled in the current environment, within its permission settings.

"Search the codebase, run tests and open a PR with the changes."



SECTION 02

Meet your two AI teammates

Both read, plan, edit, and run code — from a terminal or a desktop app.

NUS National University of Singapore

ANTHROPIC

Claude Code

An agent for project work across the terminal, IDE and desktop. It can inspect relevant files, edit code and run checks. Available actions depend on permissions.



Reads files
Undertands your project

Edits code
Makes changes directly

Runs checks
Tests, linters and more

Works where you are
Terminal, IDE or desktop

"A thoughtful coding partner that helps you plan, implement and verify."

— Claude Code

OPENAI

Codex

OpenAI's coding agent, available through local tools and cloud workflows. It can inspect a project, implement changes and verify results.



Understands your codebase
Reads and reasons across files

Implements changes
Writes and edits code

Verifies results
Runs tests and checks output

Available locally and in cloud
CLI, IDE integrations and more

"Turn your ideas into working code, with less friction."

— Codex

SECTION 02

The model vs. the harness

The model generates a response. The harness connects it to tools, context and feedback.

THE MODEL ALONE

A response from the supplied context

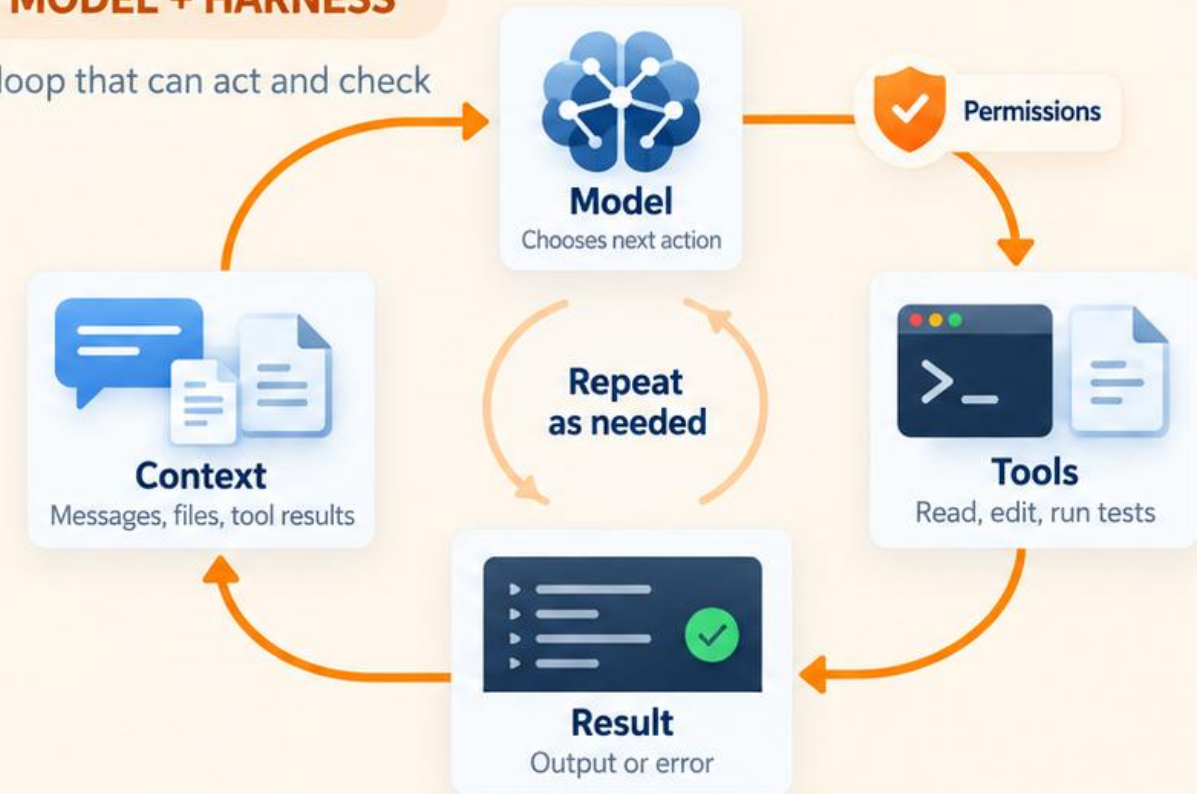


A raw model call returns output. It does not independently open files or run commands.

ChatGPT and Claude apps can also provide tools.

MODEL + HARNESS

A loop that can act and check



Claude Code and Codex combine models with this surrounding system.

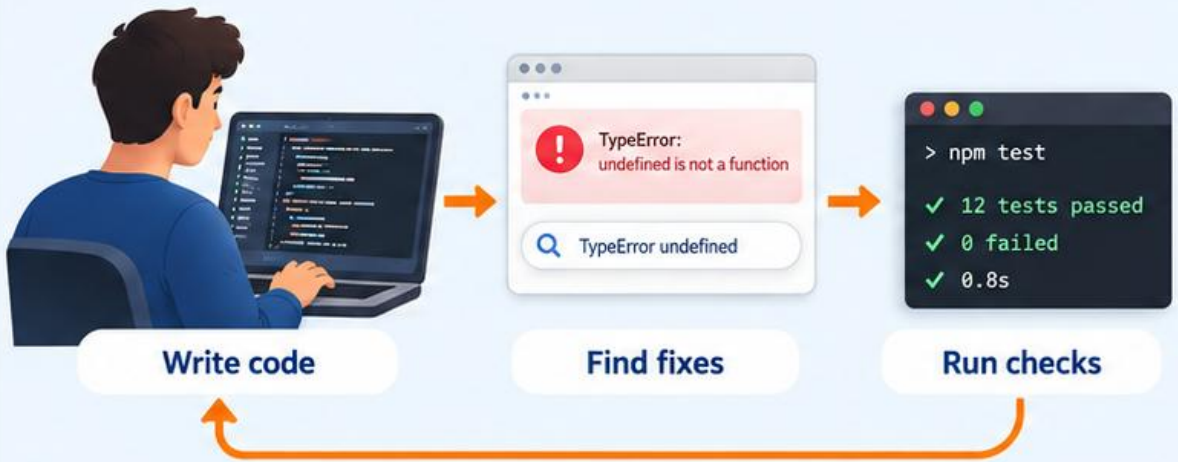
SECTION 03

From writing code to directing it

The mental shift: define the outcome, then inspect the work.

Traditional workflow

You carry out each step



- Autocomplete suggests the next token.
- You investigate errors and implement fixes.

You implement the details and verify the result.

Agentic workflow



The agent asks when the goal is unclear.

You specify the outcome and review the evidence.

SECTION 04

Getting set up

Install once, then run anywhere you have a terminal

Prerequisites

Install the tool supported on your device and sign in. Confirm your account has access before the workshop.

```
claude --version
codex --version
```

Install

Use each vendor's current installation guide. The desktop route can avoid terminal setup.

```
Current setup guides:
code.claude.com/docs/en/setup
developers.openai.com/codex
```

Run

Open your project folder and launch either tool.

```
cd "ComputeMarket Website"
claude  # or codex

Read README.md. Do not edit.
```

Today’s build: ComputeMarket

A computing marketplace frontend based on the supplied project presentation

Alice needs computing resources. Bob has capacity to offer. ComputeMarket lets them post a task, compare bids, assign a provider and review a simulated result.

Input	Project presentation, README and sample task data
Output 1	Task posting, bids and provider assignment
Output 2	Simulated result submission and a working review loop

```
$ claude  # or codex

Use frontend-design. Read the
supplied slides and README.md.
Build a marketplace frontend:
post, bid, assign and review.

Use browser-memory sample data
and clearly labeled demo credits.
Validate forms and task states.
Defer smart contracts.
No live AI or real computation.
```

SECTION 05

What happens when you hit enter

A useful workflow, with checkpoints you can inspect



Explores — Reads the proposal, project brief and frontend. Identifies missing requirements.

Plans — Chooses a small implementation plan. Ask for plan-only mode when useful.

Implements — Builds task forms, bid comparison, status views and reviews.

Verifies — Checks input limits, valid task transitions and the review loop.

Reports — Reports what works, where it lives and any remaining gaps.

SECTION 06

The project brief file

Durable project guidance, with scope determined by the tool

Keep the brief in the project root. Record the input format, checks and conventions. The tools also support scoped instructions, so keep project rules concise.

CLAUDE CODE

CLAUDE.md

```
# ComputeMarket
## Inputs
Project slides and README.md
## Outputs
Marketplace frontend with
posting, bids and reviews
## Checks
Valid budgets and delivery times.
Valid task and review transitions.
## Constraints
Browser-memory sample data only.
No wallets, contracts or payments.
```

CODEX

AGENTS.md

```
# ComputeMarket
## Inputs
Project slides and README.md
## Outputs
Marketplace frontend with
posting, bids and reviews
## Checks
Valid budgets and delivery times.
Valid task and review transitions.
## Constraints
Browser-memory sample data only.
No wallets, contracts or payments.
```

Same idea, different filename — AGENTS.md is an open convention several tools now read, including Codex.

SECTION 06

Package the workflow, don't repeat it

Turn a one-off prompt into something your whole batch can reuse

What is a Skill?

A folder with SKILL.md and optional supporting files. Both Claude Code and Codex support Skills. The agent loads the workflow when it selects the skill.

Consistent guidance: reusable design principles for layout, forms and feedback.

Less repetition: keep the procedure in a file instead of pasting it into each chat.

Shareable: version the skill with your sample inputs and checks.

```
.claude/skills/frontend-design/



frontend-design/
  SKILL.md
  LICENSE.txt

# Separate task prompt
Use frontend-design.
Read the supplied slides
and README.md.

Build task forms, bids,
status views and reviews.
Use local demo data only.
Defer contract integration.
```

SECTION 06

What the agent can (and can't) touch

Sandbox rules and approval settings control what can run

WITHIN SCOPE

- Read files the environment permits
- Run approved build & test commands
- Edit within the authorized task scope
- Read a relevant Skill or prompt file

MAY NEED APPROVAL

- Actions outside the active policy
- Edit files outside the project folder
- Destructive changes beyond the task
- Publishing or sharing beyond the request

VERIFY THE BOUNDARY

- Ignore files are not a security boundary
- Keep secrets outside demo inputs
- Check filesystem and network limits
- Review connected-app permissions

Approval behavior depends on the tool, environment and configuration. A prompt or ignore file does not replace enforced access controls.

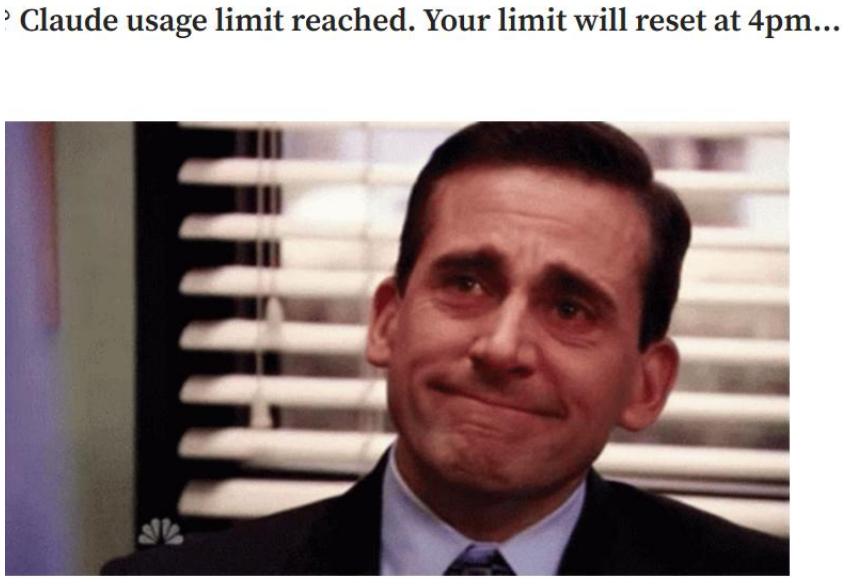
REALITY CHECK

So you've got the basics. Then, one day…

Deep in a session, marketplace half-built, demo tomorrow...

Claude usage limit reached.

Your limit will reset at 4pm.



Every power user meets this screen eventually — usually mid-flow, usually at the worst possible time.

A usage limit is a good moment to checkpoint the work. Save the plan, key decisions and commands needed to resume.

Next: choose a model and effort level, reuse a Skill, and keep the working context focused.

Models & effort — two independent dials

Model and reasoning controls vary by tool, account and release

MODEL — which weights answer you

Fast model	General model	Reasoning model	Available options
cheap & fast	*daily workhorse*	*hard agentic work*	*check your picker*

Select from the models available in your tool. Avoid hard-coding a changing catalog.

EFFORT — how hard those weights work this turn

Low	Medium	High	Higher effort
quick answers, simple tasks	everyday coding	complex reasoning	the hardest problems

Higher effort may use more time and compute. Compare outcomes on your task before raising it.

SECTION 07

Skills specify how the work gets done

A useful Skill defines inputs, a procedure, outputs and a way to check them

Analyst

Upload a CSV and get EDA, charts, and a narrative insight report — not a wall of df.describe().

Designer

Describe a screen and get a mockup or slide deck you can iterate on, not just a description of one.

Researcher

A structured literature or data synthesis — hypotheses, methodology, and a written verdict.

OUTPUTS YOU CAN INSPECT

Depending on the product, an output may appear as a file, an artifact preview or a website. Ask for the format you need and check that its controls work.

ComputeMarket produces a frontend for task posting, bids, assignments and reviews.

SECTION 07

Project slides as the starting point

The proposal explains the product. The build brief defines this tutorial.

Use the supplied computing marketplace presentation to map the requester and provider journey. The first tutorial covers screens and local interactions.

STEP 1

Inspect the proposal

Read the product idea and user journey in slides 6–8 of the supplied presentation.

STEP 2

Map the journey

Alice posts, Bob bids, Alice assigns, then reviews a simulated result.

STEP 3

Set the scope

Build forms and task states with sample data and demo credits.

STEP 4

Define the checks

Reject invalid offers and test requesting changes before approving a result.

State stays in browser memory. Smart contracts and real computation belong to later tutorials.

SECTION 07

Ready-made Skills used in this workshop

A concrete workflow from the repositories already in this deck

frontend-design

anthropics/skills / skills/frontend-design

The website demo directly uses this ready-made Skill. It guides visual direction, typography and interaction design.

Subject first: a marketplace for computing tasks

Useful controls: posting, bids and provider selection

Clear feedback: task status and form validation

Working output: a frontend students can review

```
Claude: .claude/skills/frontend-design/
Codex:  .agents/skills/frontend-design/
Each folder contains SKILL.md.
```

anthropics/skills

Official skill examples

The source of the frontend-design Skill used here. Read a skill and its license before reusing it.

andrej-karpathy-skills

Optional reading, from the original deck

forrestchang/andrej-karpathy-skills offers guidance for focused coding changes. It is not required for this demo.

The kit includes frontend-design unchanged with its license, a project brief and local sample data.

SECTION 07

Connectors — plug Claude into your tools

Give Claude access to the apps and data you already use, without copy-pasting

A connector links an agent to an external app. MCP is one way to expose tools and data. Available actions depend on the connector, account and permissions.

Google Drive & Gmail

Search docs, read files, draft email — no more pasting content in.

GitHub

Read issues and PRs, reference code, open branches.

Slack

Pull thread context, summarise channels, draft replies.

Notion

Search and read your team or personal workspace.

Microsoft 365

Word, Excel, Outlook, and Teams, connected directly.

Any MCP server

Other servers can expose domain-specific tools. Check compatibility before adding one.

SECTION 07

MCP under the hood — and how to add your own

MCP servers expose a defined set of tools and resources to a compatible agent

For a connected app, use the official setup flow for your client and provider. A server may run locally or remotely. Authenticate and check the tools it exposes before using it.

Remote HTTP

```
claude mcp add --transport http
notion \
  https://mcp.notion.com/mcp
```

Provider setup

```
Use your provider's current
MCP documentation.

Endpoints and supported
transports can change.
```

Local stdio

```
claude mcp add playwright -- \
  npx -y @playwright/mcp@latest
```

Scopes: --scope user makes a server available across projects. --scope project writes it to .mcp.json for the project.
ComputeMarket uses local sample data. No connector is required for this frontend tutorial.

SECTION 07

Hooks — automate what happens around the agent

Shell commands that run automatically at fixed points, whether or not Claude remembers to

A hook is a command you register for a lifecycle event — it fires every time, deterministically. No prompting required, no relying on the model to remember a rule.

PreToolUse

Before a tool runs — can block or modify it. Stop a risky bash command before it executes.

PostToolUse

After a tool finishes — log it, or clean up. Auto-run a formatter every time a file is edited.

UserPromptSubmit

When you submit a prompt — inject context automatically, every session, no copy-pasting.

```
~/.claude/settings.json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit",
      "hooks": [{ "type": "command", "command": "black *.py" }]
    }]
  }
}
```

Example only. Configure the matcher and formatter for your project, and verify which files it will change.

SECTION 07

Projects & Tasks

Persistent context and scheduled execution are separate capabilities

PROJECTS

Its own context, every time you open it

Custom instructions — Set how Claude should behave for everything inside this project.

Knowledge files: the product may retrieve relevant content rather than read every file on every turn.

Project context: what persists and where it is available depends on the product.

For students: one project per module, one per big assignment.

TASKS

Set it once, it runs without you

One-off or recurring — “Every Monday at 8am, check for new readings and add them to my schedule.”

Execution location varies: cloud tasks and desktop automations have different device requirements.

Check the result: a completed run can still contain missing inputs or errors.

SECTION 07

Cowork — hand off the whole job

Give Claude a goal on your desktop, come back to finished work

Where Claude Code is built for editing a codebase, Cowork is built for everyday knowledge work — research, spreadsheets, decks, organizing files. You describe the outcome, Claude plans and works across many steps, and can use the same Connectors, Skills, and Tasks you've already set up.

1

Give it a goal

“Pull last week's survey responses and build a summary deck.”

2

Let it work

The agent reads the authorized inputs, uses available tools and may need clarification.

3

Review the result

Open the resulting document, spreadsheet or deck and check it against the requested outcome.

HABITS

Three habits worth forming early

Simple practices that separate frustrated users from productive ones

Plan before you edit

Use plan-only mode when the task has important choices. For a scoped edit, give a clear target and let it proceed.

EXAMPLE

```
› Plan the changes first,
don't edit yet.
```

Watch the context window

Long sessions accumulate context. Save decisions and compact or start a new task when the topic changes.

EXAMPLE

```
/compact    /clear
```

Verify, then trust

Ask for meaningful checks: the right output, valid calculations and no missing source evidence.

EXAMPLE

```
› Test inputs and task
transitions.
```

PRO TIPS

Working like a power user

Once the basics click, context management is what separates fast users from stuck ones

CLAUDE CODE

Run parallel sessions — Separate worktrees for independent tasks, named so you can jump between them.

Delegate to subagents — A subagent in .claude/agents/ handles one job (like writing tests) and returns just a summary — the main session's context stays clean.

Dial effort to the task — Push to max for hard debugging or architecture calls; drop back down for routine edits so you don't burn usage.

CODEX

Fork instead of losing your thread — /fork branches the conversation so you can try an alternative approach without discarding the original.

Resume, don't restart — /resume picks a saved session back up — your context and history come with it.

Keep permissions appropriate to the project. Review access before changing approval settings.

Common thread: treat context as a limited resource — branch it, delegate it, or compact it, rather than making one thread do everything.

PRO TIPS · FUN FACT

Why your token bill explodes

Long working context can increase usage, latency and cost

Each turn uses the context selected by the product, such as messages, files and tool results. It may reuse cached input, compact history or retrieve only relevant content. Conversation length alone does not tell you the bill.

ILLUSTRATIVE CONTEXT GROWTH

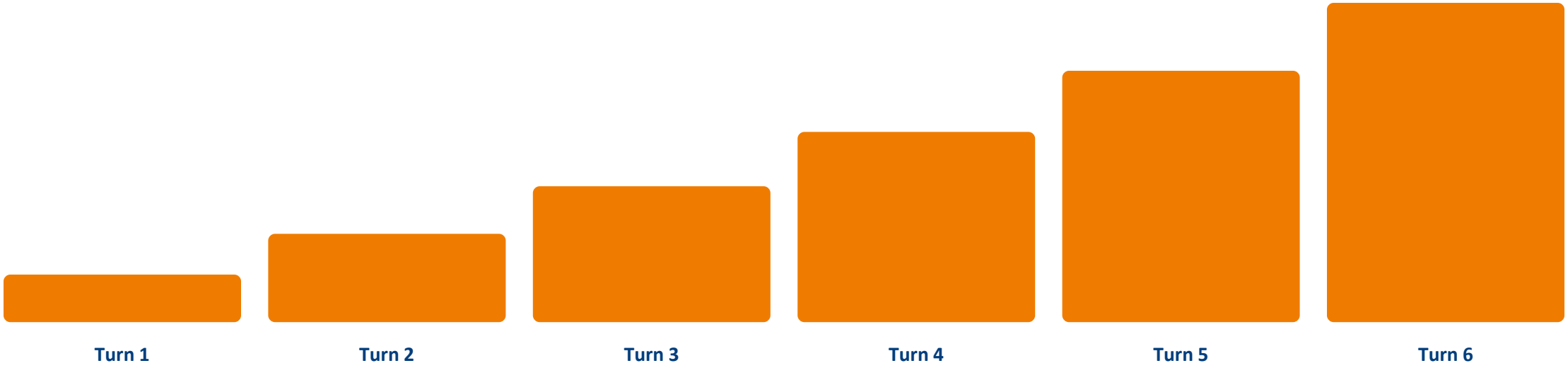


Illustration only, not a measured usage chart. The bars show accumulating context before caching or compaction. Check the tool's usage view for actual limits and billing.

PRO TIPS

Cut the bill: two playbooks

Keep context useful, and verify the effect in your tool's usage view

CLAUDE CODE

Start a fresh task when the subject changes. Keep only context needed for the new goal.

Keep CLAUDE.md concise. Put task-specific procedures in Skills that load when selected.

Use an available model suited to the task. Increase capability or effort when the result warrants it.

Delegate noisy work to subagents — Test runs and log parsing return a summary, not the raw output.

CODEX

Goal / Context / Constraints / "Done when" — Front-load these four things in the prompt — cuts wasted back-and-forth.

Durable rules live in AGENTS.md — Not repeated in every prompt.

Match effort to the task — Low for scoped edits, Extra High only for long reasoning-heavy work.

One chat per coherent task — Fork only when work truly branches — one giant chat for a whole project bloats context and degrades results.



Shorter answers can reduce output tokens. They do not remove the cost of reading large files, long histories or repeated tool results. Avoid fixed savings claims without a reproducible measurement.

Live demo: ComputeMarket frontend

A 25-minute walkthrough using local sample data. The next four slides are presenter checkpoints.

Scene 1 4 min

Read the proposal

Map the requester and provider journey. Define the frontend scope.

```
Read the project slides
and README.md. Describe
the user journey and
today's frontend scope.
```

Scene 2 6 min

Use the ready Skill

Show frontend-design, then build or open the prepared website.

```
Use frontend-design.
Build task posting, bids,
assignment and reviews.
Use local sample data.
Defer smart contracts.
```

Scene 3 15 min

Post, bid and review

Follow Alice and Bob. Check invalid offers, revisions and completion.

```
Post as Alice. Bid as Bob.
Assign a provider and
submit a sample result.
Request changes, resubmit
and approve the result.
```

Decentralized Computing Marketplace

Putting idle computing resources to work



Providers

Offer spare computing capacity

Marketplace

Connect computing supply and demand

Requesters

Access computing resources

A proposed network connecting spare capacity with computing needs.

From Idea to Prototype

Validate the idea, then build a functional frontend and backend prototype together.

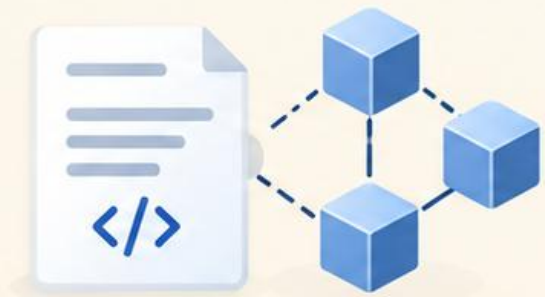
1 Validate the idea



2 Build the prototype



3 Next week



Today: working frontend + backend

Smart contracts come next

Today's build happens live, together, in this chat.

The Problem



Underutilized Resources

Millions of devices worldwide have idle computing power



Centralized Cloud Computing Limitations

High costs due to centralized control.Potential single points of failure.



Accessibility Barriers

Small enterprises and individuals struggle with affordability and scalability.

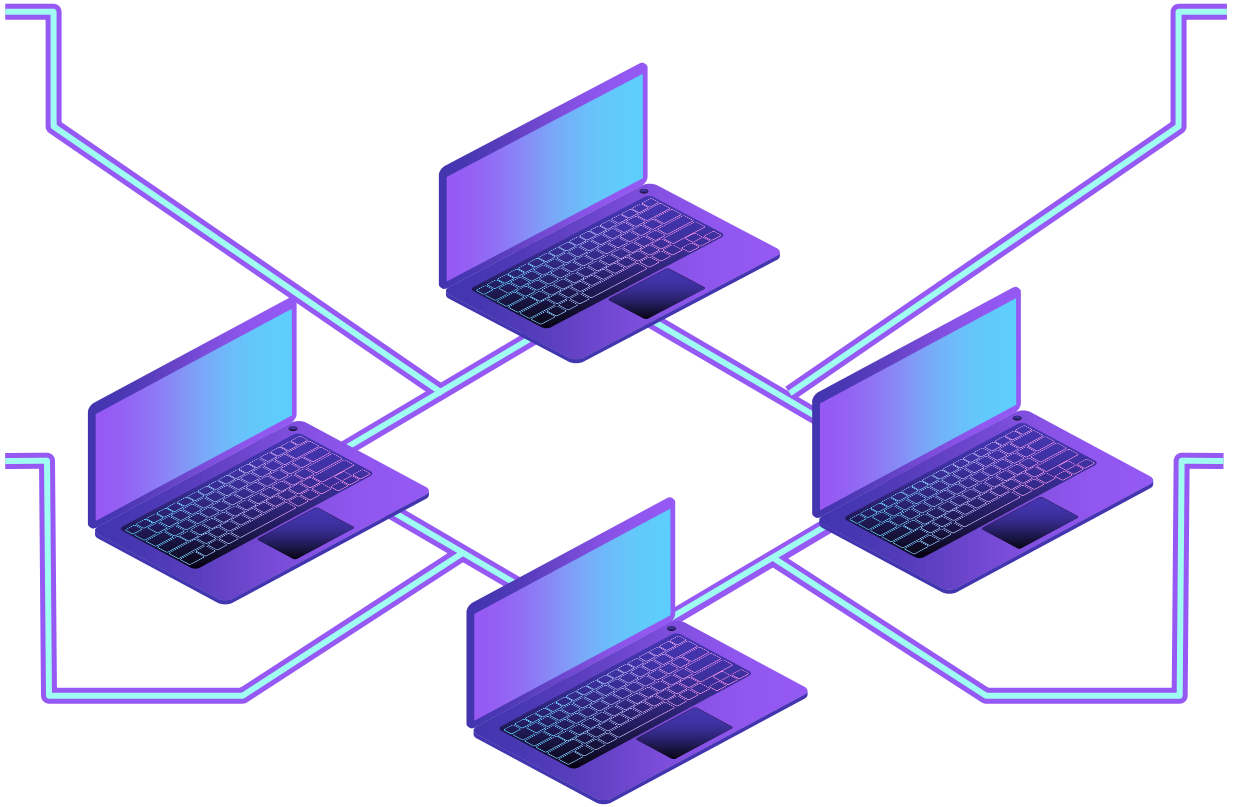
The Opportunity

Harness Idle Computing Power

Utilize global unused resources.

Democratize Access

Make computing power affordable and accessible.



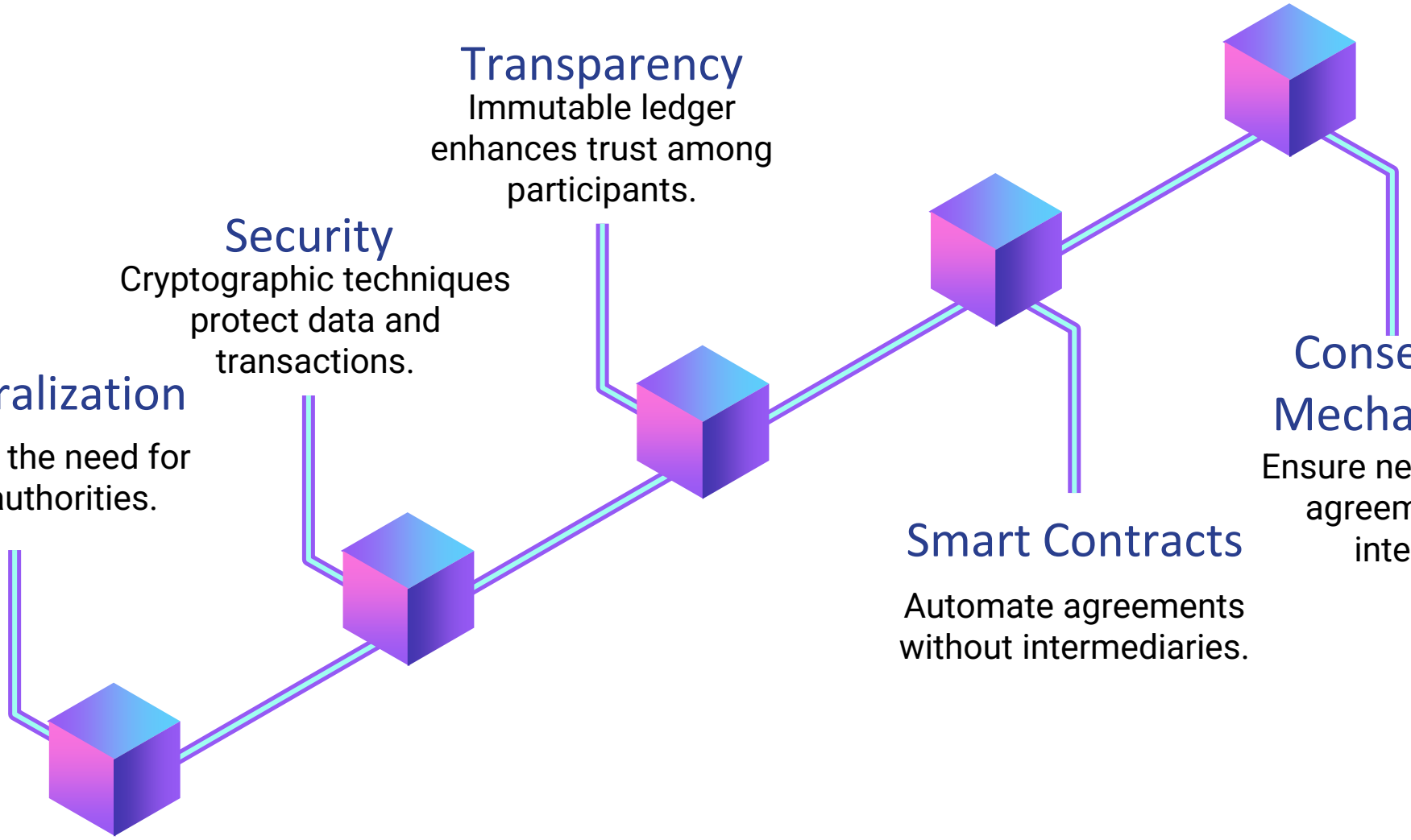
Flexible and Scalable Computing

Allow users to easily increase computing based on demand

Global Marketplace

Connect providers and clients directly.

Why Blockchain



Decentralization

Eliminates the need for central authorities.

Security

Cryptographic techniques protect data and transactions.

Transparency

Immutable ledger enhances trust among participants.

Smart Contracts

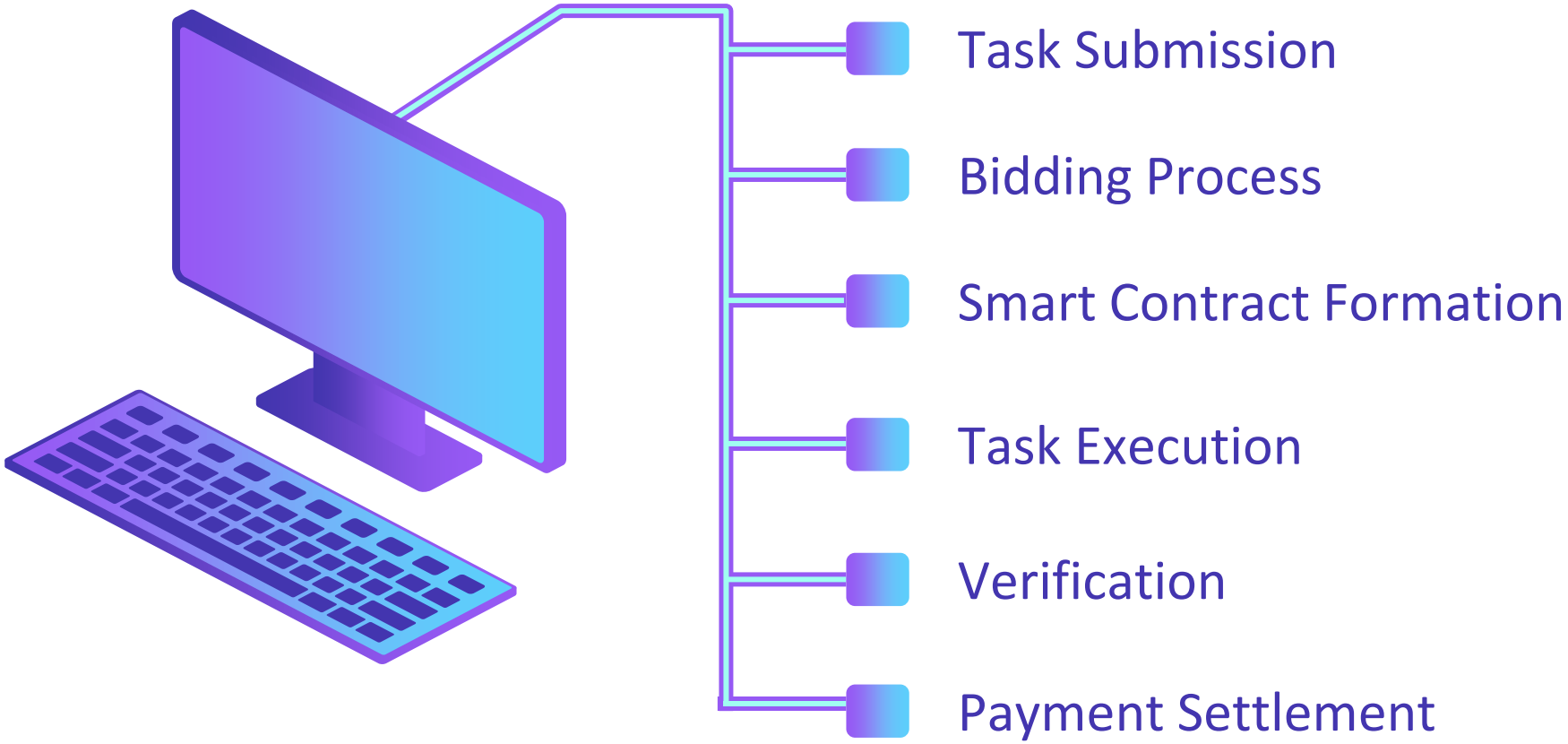
Automate agreements without intermediaries.

Consensus Mechanisms

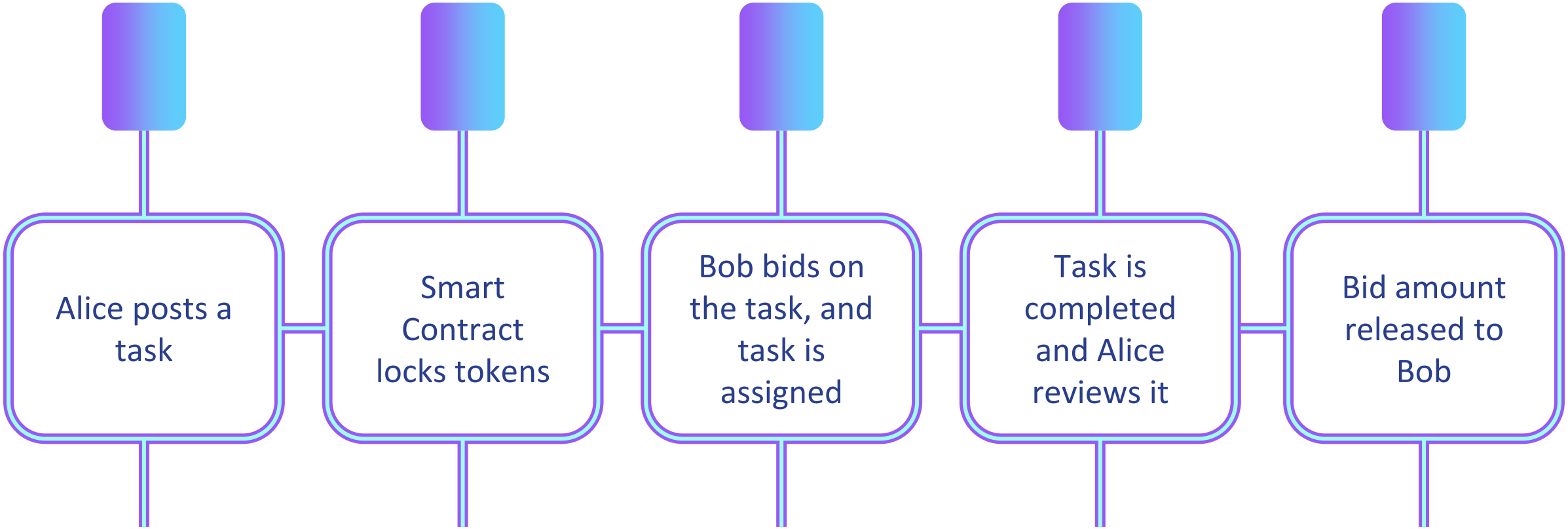
Ensure network-wide agreement and integrity.

Proposed Solution - Overview

A marketplace for trading computing resources that resides on blockchain technology



User Journey



Task submission & Bidding process



Dynamic Pricing Mechanism

Auction-Based Allocation for efficient market pricing

Reputation System

Develop reputation scores influence task assignment and pricing

Task Categorization & Specialization

Categorize tasks for appropriate allocation of resources

Task Execution – Security

Advanced Encryption Techniques

Zero-Knowledge Proofs: Verify computations without revealing underlying data

Homomorphic Encryption: Computations on encrypted data

End-to-end encryption: communications and transactions between clients and providers



Sandboxed Environments for Providers:

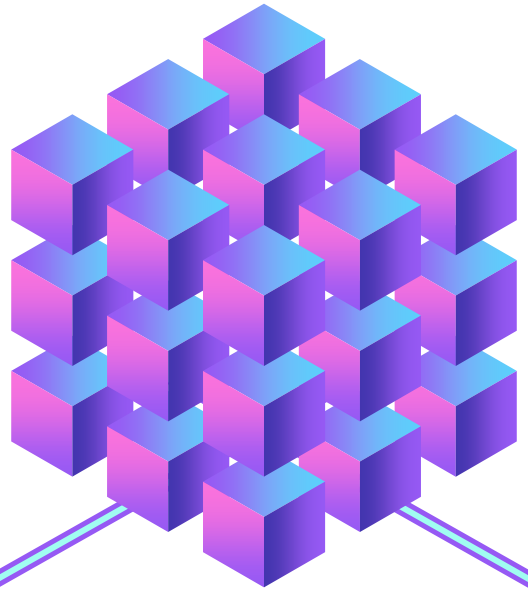
Isolated, sandboxed environments on provider machines: limits the potential damage of malicious code,
Protecting the provider from unintended consequences.

Task Execution – Scalability

Off-Chain Computation for Complex Allocations

Layer 2 Solutions

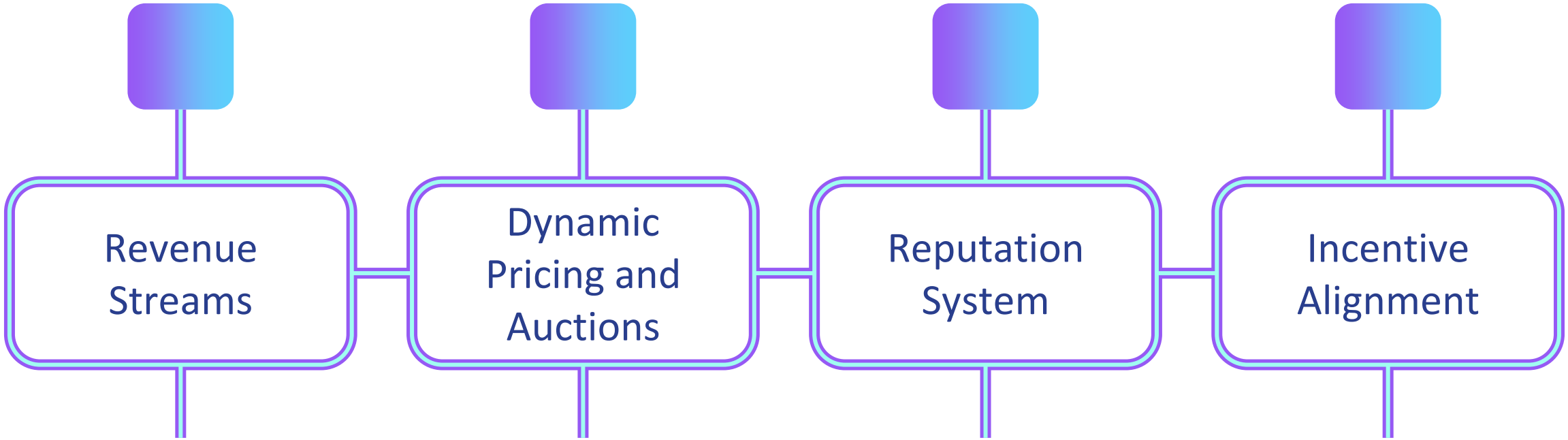
Leverage off-chain computation to enhance scalability and reduce blockchain congestion. Verifying results on-chain for efficiency.



Sharding Technology

Divides the blockchain network into smaller, interconnected partitions which can process its own set of tasks and transactions,

Business Model & Profitability



Transaction Fees:
Fees on transactions to support platform

Token Appreciation:
Increased platform usage drives demand for the native token

Efficient Markets:
Ensures fair pricing based on real-time supply and demand.

Provider Benefits:
Optimize earnings by adjusting prices according to market conditions.

Trust Building:
Reputation scores foster trust among participants.

Quality Incentives:
Providers are incentivized to maintain high-quality service

Long-Term Engagement:
Token economics encourage participants to remain active

Fair Pricing:
Market dynamics and competition help regulate costs

Skills we use

Community libraries and custom workflows for Claude Code

Community libraries



Agentic Awesome Skills

github.com/sickn33/agentic-awesome-skills

Awesome Skills

github.com/awesome-skills

Explore reusable skills and choose what fits the task.

Our custom skills

https://bit.ly/msba17sep

junior-team-skills/



junior-analyst/

Business research pipeline

Feasibility, competitor analysis, market sizing, BRD and pitch deck.



junior-designer/

UX and product design

UX research, wireframes, design systems and prototypes.



junior-database-engineer/

Data and database planning

Data audits, schema design, ERD and migration plan.



To use: copy the custom skill folders into your Claude skills folder.



https://bit.ly/msba17sep

SESSION FLOW

Four phases, one continuous thread

Each phase hands its output directly to the next — nothing gets rebuilt



1

Analyze

Junior Analyst skill reads the idea and produces a real feasibility + market read.



2

Refine

We discuss the analysis as a team and lock a final scope together.



3

Build

That scope becomes the brief Claude builds the prototype from, live.



4

Polish

Frontend design + other skills sharpen the UI while we watch it happen.

PHASE 1 · ANALYZE

The Junior Analyst skill validates the idea first

Before anyone writes a line of code, we check the idea holds up

WHAT IT DOES

A research pipeline that takes the project idea and produces a structured analysis: market read, competitive landscape, feasibility, and risks — the same rigor as a first analyst pass on a real deal.

Why now, not later

Cheaper to catch a weak assumption before four people spend an hour building around it.

WHAT WE GET OUT OF IT

- Market size & who else is building this
- Feasibility read on the marketplace mechanics
- Key risks — technical and go-to-market
- A reference document we carry into Phase 3

Checkpoint: after this, we talk as a team — tweak the idea if needed — before moving on.

PHASE 2 · REFINE

The analysis becomes the brief

Whatever we agree on here is exactly what Claude builds from next

1

Read it together

Everyone skims the Junior Analyst output — five minutes, not a debate.

2

Flag what to change

Anything that doesn't match reality gets called out now, not mid-build.

3

Lock the scope

The final version becomes the single reference doc for Phase 3.

No new document gets written here — we edit the Junior Analyst output directly and carry it forward.

PHASE 3 · BUILD

Live build: the reference doc becomes a prototype

Right here, in this chat — frontend, backend, and a simulated blockchain layer

Frontend

Task board, bidding view, and a live dashboard — the actual screens Alice and Bob would use.

Backend

A real API handling task creation, bids, and status — no mock data once it's running.

Simulated chain

Escrow, locking, and release logic written in plain code, standing in for the smart contract we build next week.

THE BRIEF WE'LL HAND OVER

```
› Here's our refined project analysis [attached]. Build a functional prototype: a task-board frontend and a backend API for posting tasks, bidding, and tracking status. Simulate the escrow/lock/release logic in plain code — no real blockchain calls, we're building the smart contract next week.
```

PHASE 4 · POLISH

Two skills, applied while we build — not after

Shown live, right where they change the outcome

Karpathy skill

Brought in as guidance for this chat: think before coding, simplicity first, surgical changes, goal-driven execution. Applied the moment the build starts, so you see the difference in how Claude plans.

Frontend design skill

Applied specifically at the UI step — task board and dashboard get intentional layout and typography instead of a generic template. Best shown as a before/after.

If a dashboard chart comes up, that's a natural third moment — worth watching for, not forcing.

SECTION 10

Where to go from here

Choose the scope you can review and verify

L1

Ask questions

Use Claude or ChatGPT for explanations and code review. Check the answer before relying on it.

L2

Small targeted edits

“Fix this function.” Single file. You read every change before saving.

L3

Multi-file features

“Add a CSV export.” The agent plans, edits several files, runs tests. You review the diff.

L4

Reusable Skills

Reuse frontend-design with a project brief and checks across other frontend projects.

Increase task scope when you can define a useful check and recognize an incorrect result.

Six things to take away

A working frontend, tested task flow and reusable design workflow

1

The model reasons, the harness acts.

Claude Code and Codex both wrap a model with tools, a loop, and permission rules.

2

Start with a project brief.

CLAUDE.md or AGENTS.md — five minutes of setup saves hours of re-explaining.

3

Skills turn a prompt into a product.

Reuse a ready Skill with the context and checks for each new project.

4

Permissions shape what can run.

Approval and access rules depend on the configured environment.

5

The working environment matters.

Choose by the files, tools, environment and review process the task needs.

6

Match scope to your ability to verify.

A useful test or source check makes the result easier to trust.



Thank You!